

# Chat export

## YubiKey, GnuPG, and rclone secrets

*Sunday 27 September 2026*

*Full working session, excluding the later request to compile a setup-guide PDF  
and excluding that guide's contents.*

This is a reconstructed transcript of the conversation: your questions in order, the answers given, and the scripts as they stood at the end of the session. A separate file, YubiKey\_GPG\_Setup\_Guide.pdf, already holds the distilled install runbook and is omitted here.

## 1. Decrypt a GPG+base64 blob into variables

**YOU**

*I encrypt passwords with `gpg --symmetric (aes256, s2k-count 65011712, armor)` piped to `base64 --wrap 0`. Make a bash script that decrypts a pre-encrypted string containing an unspecified number of `KEY=VALUE` items separated by pipes, and creates variables.*

**GROK**

Reverse the pipeline: `base64 -d`, then `gpg --decrypt` with matching loopback / no-symkey-cache flags. Split on “|”, require valid shell names, optionally decode %7C as a literal pipe. Must be sourced so assignments survive. A full-featured `decrypt_secrets.sh` was written (file, passphrase-file, dry-run, quiet). You then asked for a short embed-the-blob snippet with a PIN/passphrase prompt and almost no error checking.

### Original encrypt command (symmetric era)

```
echo $(echo "RCLONE_CONFIG_SECRET_TYPE=crypt|RCLONE_CONFIG_MYSECRET_PASSWORD=sedd" | gpg \
--symmetric --pinentry-mode=loopback --no-symkey-cache \
--cipher-algo aes256 --digest-algo sha256 \
--s2k-digest-algo SHA512 --s2k-cipher-algo AES256 \
--s2k-mode 3 --s2k-count 65011712 \
--cert-digest-algo sha256 --compress-algo none -z 0 \
--quiet --no-greeting --armor | base64 --wrap 0)
```

Warning given later: do not wrap `gpg --armor` in unquoted `echo $(...)`. That collapses armor newlines. Use `printf '%s'` instead.

## 2. “How do I download files from sources?”

**YOU**

*how do I download files from sources*

**GROK**

Interpreted mainly as rclone remotes plus generic HTTP/git. After exporting the rclone crypt variables: `rclone copy remote:path ./local-dir`, `rclone sync`, `listremotes / lsd / ls`. For URLs: `curl -fL -o file URL` or `wget`. For git: `git clone` or `raw.githubusercontent.com`. The decrypt script itself has no public URL.

## 3. Minimal decrypt snippet

**YOU**

*Make a version that is just a snippet to decrypt an encrypted string embedded in the script and prompts for passwords with minimal error checking.*

**GROK**

Source the file. Early snippet (still symmetric + loopback + pipe delimiter):

```
blob='PASTE_BASE64_ENCRYPTED_STRING_HERE'

plain=$(printf '%s' "$blob" | base64 -d | gpg \
  --decrypt --pinentry-mode=loopback --no-symkey-cache \
  --quiet --no-greeting)

IFS='|' read -ra pairs <<< "$plain"
for pair in "${pairs[@]}; do
  [[ -z $pair ]] && continue
  key=${pair%*=*}
  val=${pair#*=*}
  printf -v "$key" '%s' "$val"
  export "$key"
done
```

## 4. Environment-variable security tools

**YOU**

*Investigate environment variable security tools*

**GROK**

Env vars are a delivery mechanism, not a vault. Same-UID processes can read /proc/PID/environ; children inherit the block; logs and docker inspect leak them.

### Categories discussed

**Encrypted files:** dotenvx, SOPS (+ age / KMS / PGP), age itself, git-crypt, envsec/KMS. Closest to the homemade GPG blob. **OS keychain:** envsec (Keychain / Secret Service / Credential Manager), 1Password op run, agentsecrets (non-dumpable child). **Secrets managers:** Doppler, Infisical, Vault/OpenBao, AWS/GCP/Azure, Chamber. These do not fix /proc/environ unless the app stops using env. **Runtime hardening:** envblock (eBPF on execve), agentsecrets, secretenv, VAR=value cmd instead of export. **Scanning:** gitleaks, trufflehog, git-secrets, detect-secrets, envx, envex.

Recommendation for this job: sidecar secrets.env.enc via SOPS+age or dotenvx, or 1Password op run. The GPG snippet is fine personally but decrypt-into-the-parent-shell is the weak part.

## 5. Split the blob across lines

**YOU**

*what is best way to split blob into multilines*

**GROK**

Best: quoted heredoc + strip whitespace from the outer base64 only.

```
blob=$(tr -d '[:space:]' <<'EOF'
LS0tLS1CRUdJTiBQR1AgTUVTU0tFHR50tLS0t
ClhNRU5DLkFDQkQwLi4u
EOF
)
```

Adjacent quoted strings also work. Backslash continuations break if a space follows \. Never run tr -d '[:space:]' on raw PGP armor — that is the next error.

## 6. GPG pinentry modes

**YOU***Explain GPG pinentry modes***GROK**

gpg talks to gpg-agent; the agent talks to a pinentry program unless loopback is set.

|               |                                                 |
|---------------|-------------------------------------------------|
| ask (default) | launch pinentry GUI or TTY                      |
| loopback      | agent asks gpg; gpg reads PIN/passphrase itself |
| cancel        | fail with canceled                              |
| error         | fail with no pinentry                           |
| default       | agent default, which is ask                     |

On GnuPG  $\geq 2.1$ , `--passphrase` / `--passphrase-fd` / `--passphrase-file` need loopback (often with `--batch`). Loopback does not retry a bad secret. Use loopback for old symmetric scripts in a terminal. Use ask for YubiKey card decrypt. Never put `--passphrase` on the command line (ps sees it).

## 7. “invalid armor header”

**YOU**

*those line splitting methods give this error: invalid armor header: XMENC.ACBD0...-----END PGP MESSAGE-----\n*

**GROK**

GPG received armor with its required newlines removed. Two layers exist: inner `--armor` (must keep newlines, including the blank line after BEGIN) and outer base64 `--wrap 0` (whitespace may be stripped). Causes: `tr -d` on the PGP block; `echo $(gpg --armor)` at encrypt time; treating dummy placeholder base64 as real data; `base64 -d` on raw armor.

Starts with `LS0t` → outer base64; strip spaces, `base64 -d`, then `gpg`. Starts with `-----BEGIN PGP` → raw armor; do not strip, do not `base64 -d`.

## 8. Short encrypt snippet (still symmetric at that point)

**YOU**

*show me a bash snippet to encrypt the string*

**GROK**

```
plain='RCLONE_CONFIG_SECRET_TYPE=crypt|RCLONE_CONFIG_MYSECRET_PASSWORD=sedd'
blob=$(printf '%s' "$plain" | gpg \
  --symmetric --pinentry-mode=loopback --no-symkey-cache \
  --cipher-algo aes256 --digest-algo sha256 \
  --s2k-digest-algo SHA512 --s2k-cipher-algo AES256 \
  --s2k-mode 3 --s2k-count 65011712 \
  --cert-digest-algo sha256 --compress-algo none -z 0 \
  --quiet --no-greeting --armor | base64 --wrap 0)
printf '%s\n' "$blob"
printf '%s\n' "$blob" | fold -w 64
```

## 9. Better ways to embed secrets in a script

**YOU**

*is there a better way to embed encrypted passwords in a script?*

**GROK**

Best: do not embed. Inject at run time (op run, sops exec-env, dotenvx run). If it must live with the script: age -p is simpler than GPG. Next: dotenvx or SOPS on a .env. If GPG stays, encrypt a sidecar .env rather than a pipe-string inside the script, and decrypt only for the child — do not export into the interactive parent.

## 10. Can GPG use a YubiKey?

**YOU**

*can gpg use a yubikey*

**GROK**

Yes for OpenPGP keys, no for --symmetric. The card speaks the OpenPGP applet. Private key stays on the stick; disk holds stubs (ssb>). Encrypt to the public key; decrypt with User PIN and optional touch. gpg --card-status should show a Yubico reader and Sign/Encrypt/Auth fingerprints. Enable OpenPGP in Authenticator. Security Key series has no OpenPGP applet.

```
printf '%s' "$plain" | gpg --encrypt --armor -r 0xYOURKEYID | base64 --wrap 0
printf '%s' "$blob" | base64 -d | gpg --decrypt --pinentry-mode loopback
# later corrected: use --pinentry-mode ask for card PIN, not loopback
```

## 11. Different PINs for different uses

**YOU**

*can I set different pins for different uses*

**GROK**

Yes across applets, not inside OpenPGP. FIDO PIN, PIV PIN, and OpenPGP User PIN are independent. OpenPGP has one User PIN for sign+encrypt+auth. Split instead with touch policy per slot (ykman openpgp keys set-touch enc|sig|aut on|cached|fixed).

## 12. YubiKey software on Linux, then Gentoo

**YOU**

*where is yubikey software for linux*

**GROK**

yubico.com/support/download/. Need pcscd, ykman, Yubico Authenticator (GUI; Manager GUI is retired), gnupg+scdaemon. Flatpak: com.yubico.yubioath. Distro packages vary.

**YOU**

*I am using gentoo*

**GROK**

```

emerge --ask sys-apps/pcsc-lite app-crypt/ccid \
  app-crypt/yubikey-manager app-crypt/gnupg
emerge --ask app-crypt/yubioath-flutter-bin

# /etc/portage/package.use
app-crypt/gnupg smartcard usb

rc-update add pcscd default && rc-service pcscd start

# ~/.gnupg/scdaemon.conf (so ykman and gpg share the reader)
disable-ccid
pcsc-shared

```

Wiki: [wiki.gentoo.org/wiki/YubiKey](http://wiki.gentoo.org/wiki/YubiKey) and [YubiKey/GPG](http://wiki.gentoo.org/wiki/YubiKey/GPG).

### 13. Make a key and where to store it

**YOU**

*how do I make a key and where do I store it*

**GROK**

Generate on the computer, not as the only copy on the card. Master (certify) offline. Sign/Encrypt/Auth private subkeys on the YubiKey. Public key and stubs on the Gentoo box. Never put master or Admin PIN in git.

```

gpg --full-generate-key
gpg --export-secret-keys --armor KEYID > master-private.asc # offline
gpg --export --armor KEYID > public.asc
gpg --card-edit # admin → passwd (change 123456 / 12345678)
gpg --edit-key KEYID # key N → keytocard → slot 1/2/3 → save
gpg --delete-secret-keys KEYID
gpg --import public.asc && gpg --card-status
# want sec# and ssb> not sec / ssb

```

### 14. Will this break existing MFA?

**YOU**

*will changing these setting disrupt multifactor authentication already setup*

**GROK**

Usually no if you only touch OpenPGP. FIDO2 passkeys, OATH TOTP, and Yubico OTP are other applets. Dangerous: ykman fido reset, oath reset, disable FIDO USB/NFC, overwrite OTP slots, full device reset. OpenPGP factory-reset wipes GPG keys only.

### 15. Decrypt trouble: light on, bad password, no PIN prompt

**YOU**

*encrypt works and decrypt turns on the yubikey light but pressing the button gives bad password and no pin is prompted*

**GROK**

Classic: ciphertext on stdin plus --pinentry-mode loopback. Loopback reads the PIN from the same stdin as the message, sends empty/garbage PIN, card rejects it after touch. Fix: decrypt from a file, --pinentry-mode ask, export GPG\_TTY=\$(tty). Also confirm the message is public-key (pubkey enc packet), not still --symmetric.

**YOU**

*it decrypts with the password used to create key. it reports encrypted with cv25519 key, ID 6BDAEBAB20864DEF, created 2026-09-27*

**GROK**

GPG was using the on-disk Cv25519 secret, unlocked by the key passphrase, not the card. `gpg --list-secret-keys` must show `ssb>` for 6BDAEBAB20864DEF. `keytocard` that [E] subkey to slot 2, then `delete-secret-keys`, `import public`, `card-status`.

**YOU**

*which key id do I use to encrypt*

**GROK**

Email or primary fingerprint; GPG picks the [E] subkey. Or pass 6BDAEBAB20864DEF explicitly. Do not use [S] or [A] IDs.

**YOU**

*`printf test | gpg --encrypt --armor -r 6BDAEBAB20864DEF | gpg --list-packets | grep ... produced: encrypted with cv25519 ... public key decryption failed: Bad PIN`*

**GROK**

Encryption is correct (algo 18, that keyid). `--list-packets` still tries to unwrap the session key. Second `gpg` in the pipe has no PIN channel. Inspect with `--pinentry-mode cancel`. Real decrypt: write a file, `--pinentry-mode ask`.

**YOU**

*... PIN blocked ... pubkey enc packet version 3 algo 18 keyid 6BDAEBAB20864DEF*

**GROK**

User PIN locked after the failed empty PINs. Unblock with Admin PIN: `gpg --card-edit` → `admin` → `passwd` → `2`, or `ykman openpgp access unblock-pin`. If Admin is also locked, `ykman openpgp reset` wipes GPG keys (not FIDO) and you `keytocard` from the offline master.

**YOU**

*this decrypted with only the pin: `printf TESTTEST | gpg --encrypt --armor -r 6BDAEBAB20864DEF | base64 --wrap 0 | base64 -d | gpg --decrypt --pinentry-mode ask`*

**GROK**

Correct: secret is on the card. `ask` is right. Prefer a temp file if `pinentry` is curses on the same TTY as the pipe. Do not put `loopback` back in `gpg.conf`.

**YOU**

*it is supposed to make me press the button*

**GROK**

```
gpgconf --kill all
rc-service pcsd start
ykman openpgp keys set-touch enc on
```

off / on / cached / fixed. Then: PIN → LED → gold disc → plaintext.

## 16. Scripts updated for YubiKey

**YOU**

*modify the bash script we made this morning to use this and set as variables instead of export (attached pwdecrypt.sh with a base64 blob)*

**GROK**

Switched to --pinentry-mode ask, decrypt via mktemp so PIN is not on the pipe, set GPG\_TTY, assign with printf -v only (no export, no echo of secrets). Source the file. Without export, rclone as a child will not see the names unless you pass flags.

**YOU**

*can you use the yubikey in AWSScriptDeepv2.sh*

**GROK**

Yes. Decrypt ~/.config/rclone-aws-creds.gpg with ask before the preset menu. Re-encrypt that file to 6BDAEBAB20864DEF; the old --symmetric file will not use the card. Hide the secret in the printed command; unset keys on exit. Decrypt is before dtach so pinentry has a TTY.

```
gpg --decrypt ~/.config/rclone-aws-creds.gpg \  
| gpg --encrypt --armor -r 6BDAEBAB20864DEF \  
-o ~/.config/rclone-aws-creds.gpg.new  
mv ~/.config/rclone-aws-creds.gpg.new ~/.config/rclone-aws-creds.gpg
```

**YOU**

*make a short script to encrypt secrets from the command line and output the terminal*

**GROK**

pwencrypt.sh: encrypt to 6BDAEBAB20864DEF, print one base64 line.

**YOU**

*make both scripts so the secrets are separated by spaces and have variable number of secrets*

**GROK**

Final payload: space-separated KEY=VALUE, any count. Values cannot contain spaces. Old pipe-separated blobs are not compatible — re-encrypt.

### Final pwencrypt.sh

```
#!/usr/bin/env bash
recipient="${GPG_RECIPIENT:-6BDAEBAB20864DEF}"
if [[ $# -gt 0 ]]; then pairs=("$@"); else
    read -r -a pairs <<< "$(tr '\n\t' ' '); fi
plain=""
for pair in "${pairs[@]"; do
    [[ -z $pair ]] && continue
    [[ $pair == *=* ]] || { echo "expected KEY=VALUE: $pair" >&2; exit 1; }
    key=${pair%%=*}
    [[ $key =~ ^[A-Za-z_][A-Za-z0-9_]*$ ]] || { echo "bad name: $key" >&2; exit 1; }
    plain+="${plain:+ }$pair"
done
[[ -n $plain ]] || { echo "no input" >&2; exit 1; }
printf '%s' "$plain" \
    | gpg --encrypt --armor --quiet --no-greeting -r "$recipient" \
    | base64 --wrap 0
echo
```

## Final pwdecrypt.sh

```
#!/usr/bin/env bash
# source ./pwdecrypt.sh
blob=$(tr -d '[:space:]' <<'EOF'
PASTE_BASE64_FROM_pwencrypt_HERE
EOF
)
[[ -n ${GPG_TTY:-} ]] || export GPG_TTY=$(tty)
_tmp=$(mktemp)
trap 'rm -f "$_tmp"' RETURN
printf '%s' "$blob" | base64 -d > "$_tmp"
plain=$(gpg --decrypt --pinentry-mode ask --quiet --no-greeting "$_tmp") || return 1
read -r -a pairs <<< "$plain"
for pair in "${pairs[@]"; do
    [[ -z $pair ]] && continue
    key=${pair%%=*}; val=${pair#*=}
    printf -v "$key" '%s' "$val"
done
```

## 17. HashiCorp Vault for this use?

### YOU

*I tried hashicorp vault 10 years ago, that's a long time in people years, it seemed overly complicated for what I am doing. do you think it would be good for this type of use.*

### GROK

No. A handful of long-lived AWS/rclone secrets, one person, one Gentoo box, and a YubiKey do not justify running and unsealing a Vault cluster. Vault is for policies, audit, and short-lived credentials across many machines. Next step up if GPG feels handmade: SOPS+age, dotenvx, or pass/passage on the same YubiKey — not Vault.

## Omitted from this export

Your later request to compile the session into a YubiKey install/setup PDF, and the contents of that guide, are not reproduced here. That file is YubiKey\_GPG\_Setup\_Guide.pdf in the same folder.

*Treat key IDs, emails, and paths in this transcript as examples from a working machine. Rotate anything that should not live in a saved PDF.*